Projecte 2: Etiquetatge

Part 1: KNN i Forma

Intel·ligència Artificial

Departament de Ciències de la Computació
Universitat Autònoma de Barcelona

1 Introducció

En aquesta pràctica resoldrem un problema simple d'etiquetatge d'imatges. Donat un conjunt d'imatges d'un catàleg de roba desenvoluparem els algorismes que permetin aprendre a etiquetar automàticament imatges per tipus de peça i per color. Per simplificar la pràctica, farem servir un conjunt molt reduït d'etiquetes: 8 tipus de peces de roba i 11 colors bàsics. El sistema que desenvoluparem s'executarà sobre una imatge donada, i retornarà una etiqueta de tipus de peça de roba i una o vàries etiquetes de color. Podem veure un esquema general a la figura 1. Per fer-ho implementarem els següents algorismes vists a la teoria:

1. K-NN (`k_nn`) Mètode de classificació supervisada que es farà servir per assignar les etiquetes de tipus de roba.

2. K-means (`k_means`) Mètode de classificació no supervisada que es farà servir per trobar els colors predominants d'una imatge.

En aquesta primera part de la pràctica ens centrarem en l'etiquetatge del tipus de peça de roba i ens centrarem en la implementació de l'algorisme K-NN.

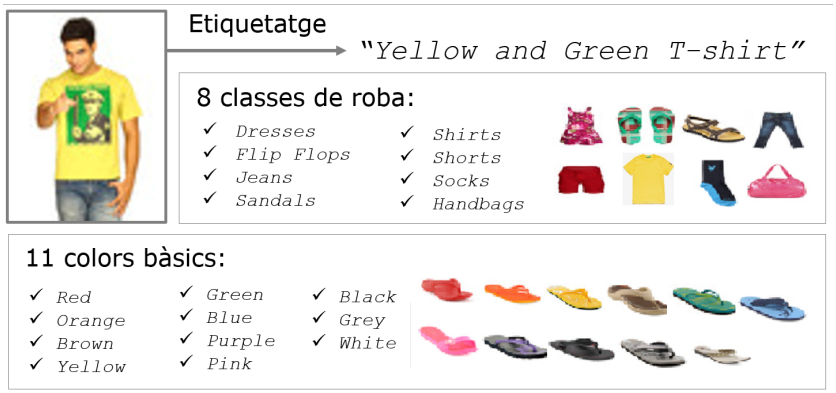

Figure 1: Objectius de la pràctica

2 Fitxers necessaris

Per a realitzar la pràctica haureu de descarregar les següents carpetes i arxius, que farem servir durant tot el projecte i en les seves diferents parts:

1. `images`: Carpeta que conté les bases de dades amb les imatges que utilitzarem.

   Dins d'aquesta carpeta trobareu:

   (a) `test`: Conjunt d'imatges que farem servir com a conjunt de test.

   (b) `train`: Conjunt d'imatges que utilitzarem com a conjunt d'entrenament per a la classificació de formes.

   (c) `gt.json`: Arxiu amb la informació del *Ground-Truth* de les imatges.

   (d) `gt_reduced`: Arxiu amb informació complementària sobre una part de les imatges que conformen el training set.

2. `test`: Carpeta que conté el conjunt d'arxius necessari per poder realitzar les proves que es demanen al test (no els heu d'utilitzar en el codi, les funcions de test els carreguen automàticament en el `setUp`).

3. `utils.py`: Conté una sèrie de funcions necessàries per a convertir les imatges en color en altres espais, principalment passar-les a gris `rgb2gray()` i obtenir la pertinença als 11 colors bàsics `get_color_proba()`.

4. `KNN.py`: Arxiu on haureu de programar les funcions necessàries per implementar KNN i per etiquetar el nom de la peça de roba.

5. `TestCases_knn.py`: Arxiu amb el qual podreu comprovar si les funcions que programeu en el fitxer KNN.py donen el resultat esperat.

6. `Kmeans.py`: Arxiu on haureu de programar les funcions necessàries per a implementar el K-means per extreure els colors predominants. (segona part del projecte).

7. `TestCases_kmeans.py`: Arxiu amb el qual podreu comprovar si les funcions que programeu en el fitxer `Kmeans.py` donen el resultat esperat. (segona part del projecte).

8. `my_labeling.py`: Arxiu on combinareu els dos mètodes d'etiquetatge i les vostres millores per obtenir l'etiquetatge final de les imatges (tercera part del projecte).

9. `utils_data.py`: Conté una sèrie de funcions necessàries per a la visualització de resultats. (tercera part del projecte)

En aquesta Part 1, ens centrarem sobretot en el fitxer `KNN.py` i `TestCases_knn.py`. Com que el K-NN és un algorisme d'aprenentatge supervisat necessitarem les imatges que hi ha al directori `Images`, i els conjunts d'aprenentatge i de test que necessitarem.

3 Preparació

En aquesta pràctica treballarem amb matrius tant per treballar amb les imatges, com amb conjunts de punts utilitzant algorismes iteratius. Per implementar aquests algorismes de manera eficient, us recomanem tenir un bon domini de la llibreria `Numpy` per a simplificar les vostres funcions.

En els següents enllaços podeu trobar alguns exercicis bàsics de `Numpy` que us poden ajudar a entendre com funciona aquesta llibreria:

- `NumPy Basic (41 exercises with solution):` https://www.w3resource.com/pyt hon-exercises/numpy/basic/index.php
- `NumPy arrays (192 exercises with solution):` https://www.w3resource.com/python-exercises/numpy/index-array.php
- `NumPy Sorting and Searching (8 exercises with solution):` https://www.w3resource.com/python-exercises/numpy/python-numpy-sorting-and-searching.php
- Altres exercicis: https://www.w3resource.com/python-exercises/numpy/index.php

En acabar amb els tutorials hauríeu de ser capaços de fer les següents operacions amb matrius `numpy`:

- Canviar les dimensions d'una matriu
- Calcular la mitjana dels valors d'una matriu
- Realitzar operacions entre matrius i vectors (ex: restar un vector a cada fila d'una matriu)
- Realitzar les dues operacions anteriors només amb certes files d'una matriu.

A la taula 1 us posem un exemple de com l'ús de la llibreria `numpy` pot accelerar l'execució d'un algorisme. Us posem dues versions d'un codi que ha de determinar els píxels que pertanyen a cada clúster per a 1000 imatges de 80×60 píxels en 6 clústers. La primera versió usa llistes de `python` i la segona usa matrius amb `numpy`, la primera versió resulta ser aproximadament 620 vegades més costosa que la versió `numpy`. En aquesta pràctica haureu de fer moltes més operacions que les que es fan en aquest codi, per tant, és essencial que ho feu d'aquesta manera si voleu fer un codi eficient.

4 Què s'ha de programar?

En aquesta primera part de la pràctica programareu l'algorisme de classificació KNN. Per a realitzar la classificació supervisada de la forma de les imatges farem servir com a característiques els píxels de la imatge després de transformar-la a escala de grisos.

Per implementar el KNN haureu de programar les següents quatre funcions:

```
#####################################################################
# Codi que determina els píxels que pertanyen a cada un de 6 clústers
# per a 1000 imatges de 80x60 píxels.
#####################################################################
import time
import random
import numpy as np
K = 6
N = 80 * 60
Iteracions = 1000

# Versió amb llistes python
###########################
llista = random.choices(range(K), k=N)
t0 = time.time()
for iteracio in range(Iteracions):
    grups = {}
    for k in range(K):
        grups[k] = [True if elem == k else False for elem in llista]
t1 = time.time() - t0

# Versió amb matrius numpy
###########################
vector_np = np.random.randint(K, N)
t0 = time.time()
for iteracio in range(Iteracions):
    grups = {}
    for k in range(K):
        grups[k] = vector_np == k
t2 = time.time() - t0

print(f'Temps per a obtenir els {K} grups per a {N} pixels, {Iteracions} vegades.')
print(f'        SENSE numpy:\t{t1} segons')
print(f'        AMB numpy:\t\t{t2} segons')
print(f'AMB és {t1/t2} vegades més ràpid que SENSE numpy')
```

L'execució retorna:

```
        SENSE numpy: 4.341734886169434 segons
        AMB numpy:  0.006999969482421875 segons
AMB és 620.2505449591281 vegades més ràpid que SENSE numpy.
```

Table 1: Exemple dels avantatges d'usar la llibreria numpy.

`_init_train`: Funció que s'assegura que la variable `train_data` de la classe KNN, la qual conté el nostre conjunt d'entrenament, tingui el format `float`, i tot seguit n'extreu les característiques. Per tant, `train_data` tindrà una mida de $N \times 4800$, on N és el nombre d'imatges del conjunt d'entrenament, i 4800 el nombre de píxels que té cada imatge 80×60.

```
def _init_train(self,train_data):
    # initializes the train data
    :param train_data: PxMxN matrix corresponding to P greyscale images
    :return: assigns the train set to the matrix self.train_data shaped
    as PxD (P points in a D dimensional space)
```

`get_k_neighbours`: Funció que pren com a entrada el conjunt de test que volem etiquetar (test_data) i fa el següent:

1. Canvia les dimensions de les imatges de la mateixa manera que ho hem fet amb el conjunt d'entrenament.
2. Calcula la distància entre les mostres del `test_data` i les del `train_data`.
3. Guarda a la variable de classe `self.neighbors` les K etiquetes de les imatges més pròximes per a cada mostra del test.

**** Pista: El càlcul de distàncies pot arribar a ser molt costo computacionalment si es fa en forma de bucle. Per fer-ho us recomanem que utilitzeu la funció `cdist`, de la llibreria `scipy.spatial.distance`, que us permetrà fer aquest càlcul de manera molt més ràpida.*

```
def get_k_neighbours(self, test_data, k):
    # given a test_data matrix calculates de k nearest neighbours
    # at each point (row) of test_data on self.neighbors
    :param test_data: array that has to be shaped to a NxD matrix
         (N points in a D dimensional space)
    :param k: the number of neighbors to look at
    :return: the matrix self.neighbors is created (NxK) as
         the ij-th entry is the j-th nearest train point to
         the i-th test point
```

`get_class`: Funció que comprova quina és l'etiqueta que més vegades ha aparegut a la variable de classe `neighbors` per a cada imatge del conjunt de test, `test_data`, i retorna un `array` amb aquesta classe per a cada imatge. Aquest `array` tindrà tants elements com punts s'hauran entrat a la funció `predict`.

```
def get_class(self):
    # Get the class by maximum voting
```

```
:return: numpy array of Nx1 elements.
   For each of the rows in self.neighbors gets
   the most voted value (i.e. the class at which
   that row belongs)
```

`predict:` Funció que pren com a entrada el conjunt de test que volem etiquetar (`test_data`) i el nombre de veïns que volem tenir en compte (`k`), busca els seus veïns utilitzant `get_k_neighbours`, i retorna la classe més representativa utilitzant `get_class`. Aquesta funció bàsicament crida a les dues anteriors.

5 Entrega de la Part 1

Per a l'avaluació d'aquesta primera part del Projecte 2 heu de completar el qüestionari Parcial del Projecte 2, Part 2: **QPar1-Proj2.1**. S'obrirà **Divendres 20 de Març a les 15:00h** i es tancarà el **Diumenge 22 de Març a les 23:55h**. És important que tingueu ben organitzades totes les funcions que feu al fitxer `KNN.py`. Durant el qüestionari, haureu d'agafar part del codi que teniu en aquest fitxer i copiar-lo a l'espai indicat per a cada pregunta.

ATENCIÓ! és important que tingueu en compte els següents punts:

1. La correcció del codi es fa de manera automàtica, per tant, assegureu-vos de copiar la funció demanada i qualsevol altra funció que aquesta necessiti.
2. El codi està sotmès a detecció automàtica de plagis durant la correcció.
3. Per a l'avaluació del vostre codi es té en compte la correctesa i l'eficiència del codi. Si les vostres funcions triguen massa, el codi les pot considerar com a errònies. Com més millorat estigui el codi, més ben avaluat serà.

Els qüestionaris de les Sessions d'Orientació i de Suport us ajuden a preparar les funcions que heu de fer. La distribució de funcions en els diferents qüestionaris és la següent:

Q1-Proj2.1 aquest qüestionari avalua entre d'altres coses, les funcions: `_init_train, get_k_neughbours, get_class i predict`.

Q2-Proj2.2 aquest qüestionari avalua entre d'altres coses la vostra capacitat d'executar l'algoritme KNN amb imatges externes.

QPar1-Proj2.1 aquest qüestionari s'obrirà des del divendres al diumenge i avaluarà totes les funcions entregades als anteriors qüestionaris, podeu fer servir aquest qüestionari per millorar-les. Aquest qüestionari és obligatori per poder aprovar, però la nota és orientativa, ja que la nota final de Codi del Projecte 2 serà la nota del Qüestionari final de la Part 3.